Instructions: https://canvas.eee.uci.edu/courses/80356/pages/projects-for-cs178-winter
Dataset: https://archive.ics.uci.edu/dataset/186/wine+quality
GitHub: https://github.com/AlistairKeiller/cs178Final/

Classifier Checklist

- [ ] kNN (range of k's, [1, 2, 5, 10, 20]) – Cindy
- [ ] logistic – Tanisha
- [ ] feed-forward (with at least 1 hidden layer) we pick 2 hidden layers – Alistair
- [ ] random forest – Teck

Report Checklist

- [ ] summary (2-4 sentences)
- [ ] data description
  - [ ] description
  - [ ] 1 figure
  - [ ] 1 paper
- [ ] classifiers
  - [ ] 1-2 sentence per classifier
  - [ ] hyperparameters tuned
  - [ ] libraries used
- [ ] experimental setup
  - [ ] metrics
  - [ ] how data was split
  - [ ] how hyperparameters were selected
- [ ] experimental results
  - [ ] interpretation of results
  - [ ] table/figure
  - [ ] comparison of performance
- [ ] insights
  - [ ] summary of what we learned
  - [ ] any surprising results
  - [ ] "error" analysis
    - [ ] weaknesses of the classifiers & comparison with each other and humans
    - [ ] strengths of the classifiers
    - [ ] (optional) speculation on performance on real world data
- [ ] contributions
  - [ ] 1-2 sentence for what each person did
- [ ] link to github

An Analysis of Machine Learning Approaches on the UCI Wine Dataset

Teck Ho, Alistair Keiller, Tanisha Singh, Cindy Zhang

I. Summary

In this paper, we compare the efficacy of kNN classifiers, logistic classifiers, neural networks, and random forests when predicting wine quality. We also discuss optimization [INSERT PROJECT IDEA HERE].

II. Data Description

The dataset used in this paper is Wine Quality from the UCI Machine Learning Repository. This dataset includes 11 features across 4898 data points collected from red and white vinho verde wine samples. Our goal is to predict wine quality, as tasters rated it between 0 and 10.

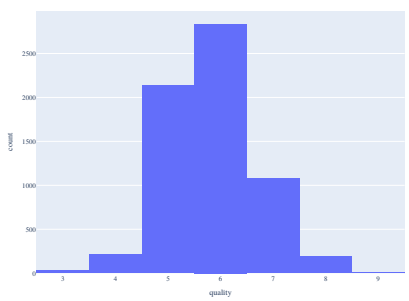

Fig. 1. Distribution of quality as rated by tasters between 0 and 10

Based on Fig. 1, we observed the quality data to be approximately normal, with a mean of 5.8 and a std of 0.87. As a result, there are no instances of quality 0 through 2 or 10, and there are very few instances of quality 3, and only 5 instances of quality 9. Based on this analysis, we can remove the bins 0-2 and 10 from our models output range, since we have no training data on those qualities. We then computed the correlation matrix.

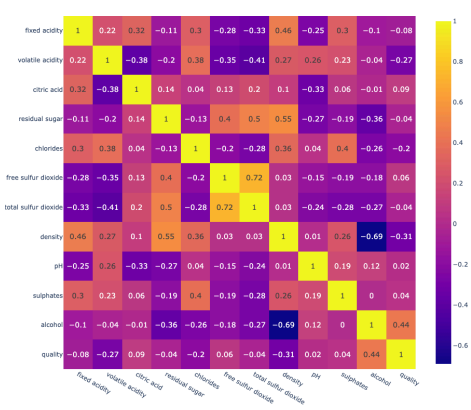

Fig. 2. Correlation between features

Using Fig. 2, we determined that alcohol content was the feature most correlated with quality, with a correlation of 0.44. We then plotted alcohol against quality and color.

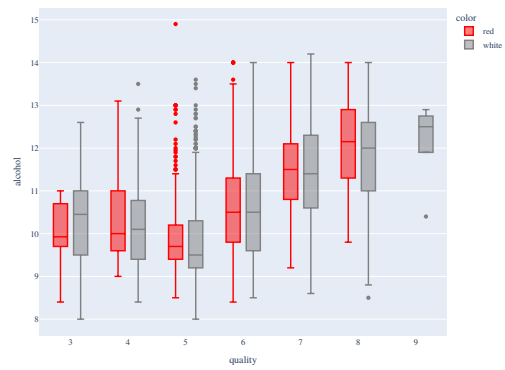

Fig. 3. Alcohol vs color and quality

Reviewing Fig. 3, we determined that low alcohol quantity is correlated with mediocre wine, low to medium alcohol quantity is correlated with low quality wine, and high alcohol quality is correlated with high quality wine.

In order to interpret models, we ideally want two features that are highly correlated with quality but not with each other. While density is the feature with the second highest correlation to alcohol, it is also highly correlated with alcohol. Therefore, instead we will use volatile acidity, with correlation -0.27.

III. Classifiers

A. kNN

kNN classifiers make predictions on points based on majority vote from their k nearest neighbors. For our implementation, we used `KNeighborsClassifer` from `scikit-learn`. The hyperparameters we tuned were the set of features used and the parameter k. To choose a set of features, we used the `itertools` library's `combinations` function to loop through all combinations.

B. Logistic

Logistic regression classifiers make their predictions based on the output of feeding input data through a logistic function. When using binary data, this is traditionally any continuous function, such as a sigmoid. With multiclass data, functions like softmax are more commonly applied. The classifier trains to minimize its loss, calculated as cross by constantly modifying its weights according to the inputs.

C. Neural Network

Neural network classifiers make their predictions by taking an input vector, feeding it through layers by using matrix multiplication, addition, and activation functions, and then then final layer is the output layer, which has the resulting output. In our case, we have 7 output nodes corresponding to each quality class (3-9). We implemented a Neural Net in PyTorch, which provides better training speed, more information about the training process, and the ability to customize the models. For example, we use dropout layers in training as a method to prevent overfitting.

D. Random Forest

Random Forest classifiers generate predictions by combining multiple decisions trees. To implement the random forest algorithm, using `RandomForestClassifier` from `sklearn.ensemble` allows the model to become more robust. Increasing the number of trees improves the accuracy score of the data by running `n_estimators`. `max_depth` enables deeper trees to expand the complexity as it produce more splits.

IV. Experimental Setup

We split our data into 20% testing, 20% validation, and 60% training. Our goal is to predict the wine quality, so we split each quality value into its own class. In order to tune or models, we plotted validation accuracy for each set of hyperparameters in a graph. We will use more detailed metrics in the results, but we decided to tune off of accuracy since it is the single metric most representative of a models usefulness.

We also find later that the model overfit to the disproportionate amount of data in classes 5 through 7. This helps improve accuracy on testing, but would result in the model very rarely giving high or low scores. In order to create a set of models that helps counteract this, we stratified the data to have an equal proportion to train and test on. Since there were very few data points to sample from amongst the extremes, we made bins for classes 3-4, 5-6, and 7-9. To stratify the data, we randomly sampled without replacement.

A. kNN

To get the best kNN classifier, we trained models against different sets of features and against different values of k.

Index	Feature Name
0	fixed_acidity
1	volatile_acidity
2	citric_acid
3	residual_sugar
4	chlorides
5	free_sulfur_dioxide
6	total_sulfur_dioxide
7	density
8	pH
9	sulphates
10	alcohol
11	color

Fig. 4. Each feature name and its corresponding index.

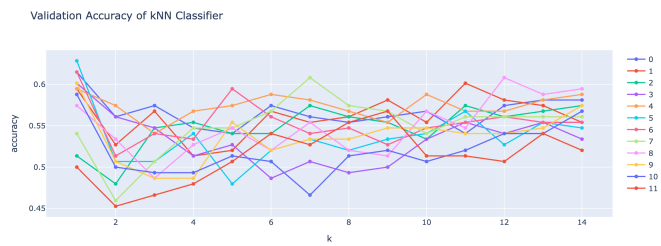

Fig. 5. Validation accuracy of kNN models when removing 1 feature over different values for k.

As a sample of what the training results were, Fig. 5 shows that removing certain features individually like feature 5 results in higher accuracy. Due to how many combinations of sets of features there are and the low readability of the graphs it results in, we are omitting a visualization of the accuracy curves for all the hyperparameters tuned. The hyperparameters that resulted in the best validation accuracy at 72.30% used features [0, 1, 2, 5, 6, 10, 11] and k = 5.

B. Logistic

C. Neural Network

Well chosen hyperparameters are paramount to a accurate Neural Network. The first hyper parameter we tune is the hidden layer sizes. Given that the dataset is small, we can assume that we will want a relatively small network to combat overfitting. An initial guess is that the network should be 16, 32, 64, or 128 nodes, and either 1 or 2 hidden layers.

```
initialize accuracy_history
for hidden_layers in possible hidden_layers
    initilize accuracy_list
    for train_size in train sizes
        sample train_size points from train_data
        train model on sample
        evaluate model accuracy on test and validation
        record accuracies in accuracy_list
    append accuracy_list to accuracy_history
```

The above psudocode allows us to generate a learning curve for both training and validation accuracy as the model gets more training data, for each possible hidden layer configuration. This is faster than sklearn's learning_curve, sinec it uses validation accuracy rather than cross validation.

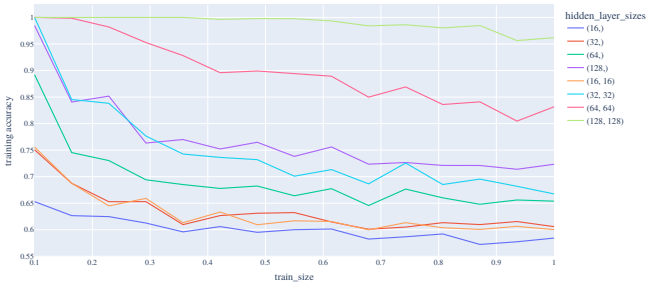

Fig. 6. Training accuracy across hidden layer sizes

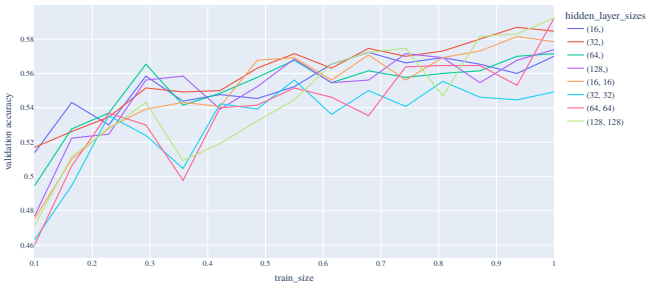

Fig. 7. Validation accuracy across hidden layer sizes

Fig. 6 shows the training accuracy for each set of hidden_layer_sizes as the size of the training set increases. All models are able to get nearly 100% accuracy when the training size is small. However, as the training size grows, the small models fail to overfit to the training data, resulting in a lowered training accuracy. The larger models, on the other hand, are able to overfit to the training data much more, making them keep a higher accuracy throughout their training.

Fig. 7 shows the validation accuracy for each set of hidden_layer_sizes. For small train_size, the smaller models perform much better, since they overfit to the training data less than the large models. However, as train_size grows, the larger models overfit less, and bridge the gap with the smaller models. With all of our training data, the largest models with size $(128, 128)$ and $(64, 64)$ are the best. This is surprising, since that is relatively large for a small dataset. Moving forward, we will use hidden_layer_sizes $(128, 128)$ instead of $(64, 64)$ since regularization and dropout aim to further reduce overfitting, they have a larger potential to improver accuracy on the larger model.

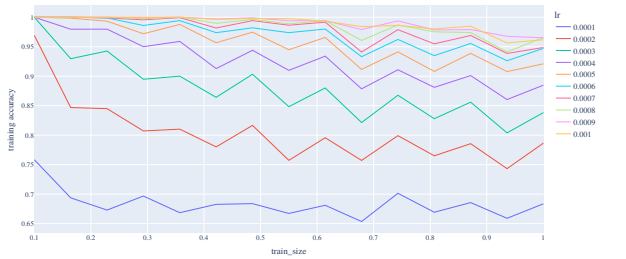

Fig. 8. Training accuracy across learning rates

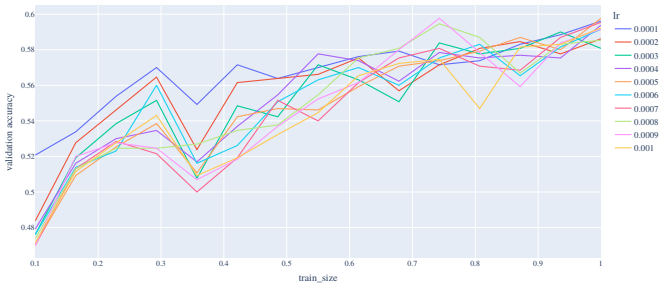

Fig. 9. Validation accuracy across learning rates

From Fig. 8 shows the training accuracy for each learning rate. The learning rate is a coefficient used in training to determine the size of each gradient decent step. We find that a learning rate greater than 0.0005 is helpful to fully fit to training data, but beyond that there is very little difference.

From Fig. 9, we find that a small learning rate is helpful with small train_size, in order to prevent overfitting to the data. However, with train_size=1, the differences are negligible, so we will just go with the highest validation accuracy of 0.005.

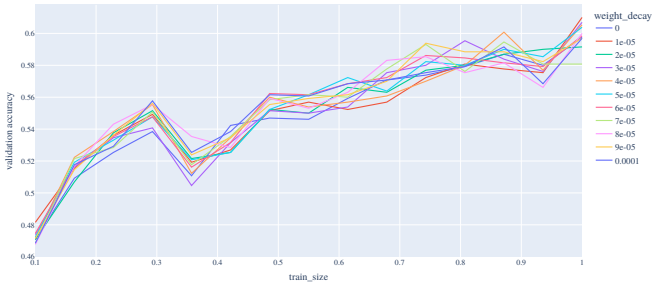

Fig. 10. Validation accuracy across weight decay

Fig. 10 shows the validation accuracy for each weight decay. The weight decay is a type of L2 regularization, which punishes large weights and biases, therefore reducing model complexity and overfitting. The weight decay with the highest validation score is $1 \cdot 10^{-5}$.

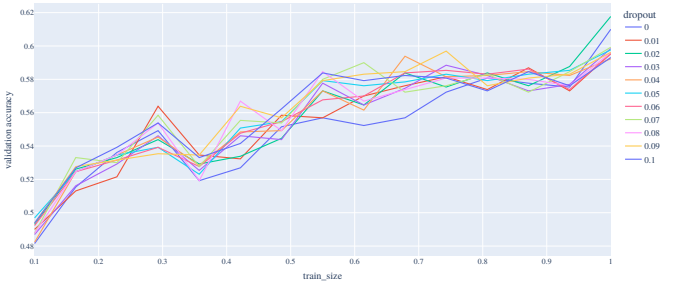

Fig. 11. Validation accuracy across dropouts

Fig. 11 shows the validation accuracy for each dropout. A dropout layer is a layer that will randomly drop neurons while training. This forces the network to become more robust, and reduces overfitting. Small values of dropout are helpful, but too large a value deceases our validation accuracy. The dropout with the highest validation score is 0.02.

Combing our results, we have a model with 2 hidden layers of size 128, a leanring rate of 0.0005, a weight_decay of 0.00001, and a dropout of 0.02. This results in a validation accuracy of 0.6177. But if we tune all the values at once we can do better. Based on all our findings, I created an optuna hyperpamater optimization run. Since our model benefited so much from being larger, I gave it a much larger model size, with either 128, 256, or 512 nodes per layer and 1-4 hidden layers. I gave it a learning rate range of $(10^{-4}, 10^{-2})$, a dropout range of $(0, 0.3)$, a weight decay range of $(10^{-6}, 10^{-2})$, and a batch_size range options of 128, 256, 512, 1024, or 2048. The optimal values were a 2 layer network with 512 node layer sizes, a learning rate of 0.002, a dropout of 0.06, a weight_decay of 0.0004, and a batch_size of 128. This results in a validation accuracy of 0.64.

D. Random Forest

V. Experimental Results

TABLE I
WEIGHTED METRICS ON TESTING DATA

Classifier	Accuracy	Precision	Recall	F1
KNN				
Logistic				
NN	0.619231	0.6178	0.619231	0.615638
RF				

VI. Insights

As mentioned in Experimental Setup, we needed to bin and stratify the data in order to assure there was an equal representation of each class. Without this adjustment, we noticed that our models tended to predict less of the qualities outside 5-7. In one extreme case, logistic regression simply did not predict any points outside 5-7. This was less prominent in other models, but it still has an effect since it gives more weight to classes 5-7. Another observation we made is that the final testing accuracy is quite low compared to some of the state of the art results on other datasets. One interpretation of this could be that humans don't have a consistent way of rating wine quality.

VII. Contributions

I would like to thank us for making this project. It was very hard. Donuts would be appreciated.

References